

1. Introduction

The project problem is a binary classification task focused on distinguishing between human-generated and machine-generated text. There are two datasets for training, each containing text samples that fall into one of two domains. The challenge lies in building a predictive model that can accurately classify text samples as either human-generated or machine-generated without knowing the source domain(s). Additionally, domain 2 presents a label imbalance issue with significantly fewer human-generated samples, which needs to be addressed during model training. The goal is to create a model that performs well on a test set in the Kaggle competition - showcasing its ability to generalise across different domains.

2. Final approach

Data pre-processing: A hybrid sampling approach was devised to address the label imbalance. Upon analysis of dataset 2, it was found that machine labelled “model”s 4 and 5 were undersampled; also, there is a significant imbalance between machine/human labelled texts. To counter this, the initial step undersample a percentage of the data, excluding “model”s 4 and 5, to help prevent classifiers from becoming biased towards the majority “model”s. Afterwards, oversampling is applied via SMOTE which creates synthetic samples whilst trying to preserve data patterns. Furthermore, applying SMOTE without the undersampling resulted in 25500 samples - compared to the approximate 21000 samples from undersampling + SMOTE. A more balanced dataset size from both domains encourages models to better generalise/prevent overfitting across the 2 domains.

Convolutional Neural Network: The final approach consisted of combining two CNNs. The first CNN had the aim to classify which domain a data sequence was generated from. Sequences were divided into 4 classes: 0 - machine domain 1, 1 - machine domain 2, 2 - human domain 1, and 3 - human domain 2. The first network consisted of a Sequential model from the Keras API. It had the following architecture: embedding layer (input), convolution layer with ReLU activation to capture local patterns in data and reduce overfitting risks, max pooling layer with ReLU to achieve robustness against variations and distortions in the input data., and dense layer with ReLU activation, output layer with softmax activation. The second CNN model processes two distinct inputs class 0-3 and sequence and combines them for binary classification. It consists of one “text” branch with an architecture of embedding layer, convolutional layer, max pooling layer, and one “domain” branch with an embedding layer and a global max pooling layer. These were then concatenated and followed by a dense layer with ReLU activation and output layer. ReLU activation functions were applied to avoid the vanishing gradient problem and for its computational efficiency, making the CNN train faster allowing more trials for tuning the network and the data preprocessing. Embedding layers were

applied to reduce dimensionality to achieve higher computational efficiency and capture underlying patterns. The model was trained using the correct models for each sequence. For the prediction of data sequences a model of the sequence was first predicted in multilabel CNN. Next the label of the sequence was predicted using the binary CNN using the predicted class 0-3.

This method scored highest in accuracy when applying it to test data. The structure of the second CNN was implemented to handle the domain differences. A potential problem with it is that the double classifying possibly adds in extra noise into the model and that the second CNN relies on the multilabel predictor to have good accuracy.

3. Alternatives Considered

Gradient boosting: XGBoost was considered as it is advantageous for text classification due to its ensemble learning, robustness to imbalanced data, and its ability to handle non-linearity and learn patterns. However, the implementation was ultimately unused, primarily due to its sensitivity to the domain shift which was unaddressed, and the availability of alternative approaches that are better suited, such as the neural networks or ensemble methods implemented.

Recurrent neural network (RNN): A RNN with LSTM was implemented, as it has benefits in handling sequential data and effective NLP capabilities - capturing contextual information and dependencies within text data (Manaswi, 2018). The implementation adhered to established best practices for effectively employing RNNs, and had the following architecture - an embedding layer, a dropout layer, a LSTM layer with tanh activation, another dropout layer, and a final dense layer with sigmoid activation for binary classification. Weights were initialised with Xavier's initialiser, and biases for the output layer set to 0. The L2 Kernel regulariser was used for applicable layers, and the model uses binary cross-entropy loss with the Adam optimizer and gradient clipping. However, it was not used in this case - despite LSTM learning sequential data well, it may have been too complex for the task - even with a basic model architecture, regularisation techniques and tuned hyperparameters, it still overfitted very quickly.

Ensemble learning: Ensemble learning was implemented to try to address the domain shift problem. The underlying idea was to train different models on the different domains, then let each model predict the testdata, and finally combine the predictions of each model. The base models that were incorporated into the ensemble model were logistic regression (LR), random forest (RF) and support vector machines (SVM). These three methods were chosen as they are diverse, which has proved to work well for ensemble implementation (Riccio, 2023). The value vectors were represented using TF-IDF vectors. To tackle data domain issues, logistic regression models were trained on each dataset, and their predictions of the AI-model were integrated as a feature into the TF-IDF vectors. In order for SVM to be time efficient, the TF-IDF vector was reduced using PCA, and normalised. For evaluating predictions from different models, the approach encompassed majority voting, which identifies the most frequent prediction for each data point. Additionally, as an alternative, a stacking approach was implemented, where base model predictions served as input for a logistic regression meta-model. This meta-model was trained to generate final predictions by learning from the contributions of individual based models. The training dataset for the meta-model was dataset 3, as it was assumed to be the most similar to the kaggle test dataset.

4. Discussion

The below table shows the accuracy results of different models on a 20% test split on the datasets provided:

Method	CNN	RNN	XGboost	3x LR	3x RFs	3x SVMs	3x LR+ 3x RF	3x LR+ 3x RF+ 3x SVM
Dataset 1	0.96	0.94	0.90	-	-	-	-	-
Dataset 2	0.82	0.84	0.87	-	-	-	-	-
Combined Dataset	0.94	0.90	0.85	0.88	0.90	0.60	0.91	0.85

When assessing the test data, the best individual and ensemble models show high accuracy and it was expected that they perform similarly in the competition. However, during training validation, the models outperformed the Kaggle test dataset, hinting at a likely domain shift as the underlying reason for this contrast.

The two domains significantly influenced method accuracy. To address this, an ensemble approach was employed, as models trained on distinct domains can complement each other. The expectation was that combining LR and RF, both strong single models, would yield even better results. This was true for the combined dataset (see Table). However, the SVM's individual poor performance had a negative impact on the combined model. SVM's poor performance may stem from PCA reduction and its inherent sensitivity to imbalanced, noisy data (Khoong, 2021).

For the RNN, another variant with a Domain Adversarial Neural Network (DANN) architecture (gradient reverse layer, dense layer with ReLU activation, and final dense layer with sigmoid activation) was implemented, to address the domain shift. However, due to limited computational resources and time constraints, the model architecture (both the DANN and RNN) was unable to be tuned/improved to be capable of capturing sufficient domain relevant information and patterns to be useful, as the domain label remained a 50% accuracy throughout training.

The combined CNN approach where a domain branch was implemented gave the CNN a categorical aspect, likely adapting it better to shifts in domain. However, one problem that became evident was its sensitivity to label imbalances. The multilabel CNN network showed low precision for class 1 and low recall for class 3 (the human samples). This can likely explain the differences in accuracy on our test data where the human samples from domain 2 were underrepresented and the kaggle test data with an equal amount of each class.

Reflecting on the project, one area for improvement was accounting for domain-specific features in the Kaggle test data, which might have contained unique patterns not considered during model development. Domain adaptation techniques like domain-invariant feature extraction could have been crucial. Additionally, overfitting in models trained on partially synthesised data could be attributed to issues like noise in synthetic samples, suboptimal feature engineering, regularisation, and hyperparameter tuning. To mitigate this, refining data generation and applying appropriate regularisation while ensuring cross-validation for accurate performance assessment could have been beneficial.

References

Khoong, W. H. (2021, August 29). *When Do Support Vector Machines Fail?* Medium.

<https://towardsdatascience.com/when-do-support-vector-machines-fail-3f23295ebef2>

Riccio, D. (2023, January 2). *Understanding the Importance of Diversity in Ensemble Learning*. Medium; Towards Data Science.

<https://towardsdatascience.com/understanding-the-importance-of-diversity-in-ensemble-learning-34fb58fd2ed0>

Manaswi, N.K. (2018). RNN and LSTM. *Deep Learning with Applications Using Python*, pp.115–126.

doi:https://doi.org/10.1007/978-1-4842-3516-4_9.